



UNSW
SYDNEY

COMP4141

Theory of Computation

Lecture 9: Turing Machines

Administrivia

- Formatif tasks ongoing
- Tutorial solutions out asap

What is Computation?

So far we have seen various models of computation:

- Deterministic Finite Automata
- Non-deterministic Finite Automata
- Pushdown Automata
- Regular expressions
- Context-Free Grammars

But they do not fully capture “computation”.

What is Computation?

So far we have seen various models of computation:

- Deterministic Finite Automata
- Non-deterministic Finite Automata
- Pushdown Automata
- Regular expressions
- Context-Free Grammars

But they do not fully capture “computation”.

Question

What more do we need?

Outline

History

Turing Machines

Turing Machine Variants

Enumeration

Closure Properties

Encodings

The Universal Turing Machine

Question

What is an algorithm?

Hilbert's 10th problem (1900)

Problem

Devise an algorithm that tests whether a polynomial has an integral root.*

Example

$p(x, y, z) = 6x^3yz^2 + 3xy^2 - x^3 - 10$ has a root at
 $(x, y, z) = (5, 3, 0)$.

Hilbert apparently assumed a “process with a finite number of operations” existed – it just needed to be found

Effective computability

Various algorithms for computing certain problems effectively were known, but a general definition of “effectively computable” that could distinguish the computable and the non-computable was sought.

Various formalisms have been proposed:

- Turing machines (Alan Turing 1936)
- Post systems (Emil Post)
- μ -recursive functions (Kurt Gödel, Jacques Herbrand)
- λ -calculus (Alonzo Church, Stephen C. Kleene)
- Combinatory logic (Moses Schönfinkel and Haskell B. Curry)

Church-Turing thesis

Surprisingly,

Theorem

All these formalisms are equivalent: a problem is decidable by one approach if and only if it is decidable by any other.

Church-Turing thesis

Surprisingly,

Theorem

All these formalisms are equivalent: a problem is decidable by one approach if and only if it is decidable by any other.

This observation strongly supports the idea that there is only one form of computation:

Church-Turing Thesis

Turing Machines define computability.

Hilbert's 10th problem (1970)

We now know:

Theorem (Matijasevich, 1970)

Hilbert's 10th problem is algorithmically unsolvable. That is, there is no algorithm that can solve the (general) problem.

Outline

History

Turing Machines

Turing Machine Variants

Enumeration

Closure Properties

Encodings

The Universal Turing Machine

Turing Machines

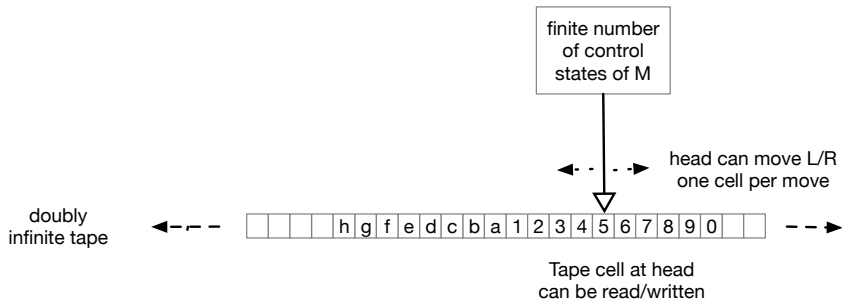
Finite control with a *tape* which is used for input and for unbounded storage. There is a “head” that can read/write the tape.

- Tape is infinite in both directions.
- There is a special “blank” symbol ($\sqcup$).
- All but a finite number of positions are blank at any given time.

Definition

A *move* of a TM is based on the current state and the symbol currently being scanned, and it changes state, writes a new symbol, and moves left or right.

Turing Machines



Formal definition

A TM is a tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ with components:

- Q : states
- Σ : input symbols, not containing the *blank symbol* $\sqcup$
- $\Gamma \supseteq \Sigma$: tape symbols, containing $\sqcup$
- $\delta : Q \times \Gamma \longrightarrow Q \times \Gamma \times \{L, R\}$
- q_0 : start state
- q_{accept} : accept state
- q_{reject} : reject state ($\neq q_{\text{accept}}$)

TM configurations

A *configuration* for a TM is a triple $(u, q, v) \in \Gamma^* \times Q \times \Gamma^*$ typically written uqv .

It represents the situation in which

- q is the current state.
- The tape content is uv
- the head is at the first symbol of v
- The infinite string of blanks to the left and right of the non-blanks uv are suppressed.

Special TM configurations

For $w \in \Sigma^*$, configurations of the form $q_0 w$ are called *start configurations*.

Configurations of the form $uq_{\text{accept}}v$ are called *accepting configurations*.

Configurations of the form $uq_{\text{reject}}v$ are called *rejecting configurations*.

NB

Various other first definitions of TMs exist in the literature but it is usually easy to move between the different formats.

Language of a TM

Definition

For $a, b, c \in \Gamma$, $u, v \in \Gamma^*$, and $q, q' \in Q$.

Configuration $uaqbv$ yields $uq'acv$ if $\delta(q, b) = (q', c, L)$.

Configuration $uaqbv$ yields $uacq'v$ if $\delta(q, b) = (q', c, R)$.

Definition

A TM $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ *accepts* input $w \in \Sigma^*$ if a sequence $C_1, \dots, C_n$ of configurations exists, where

- 1 $C_1 = q_0 w$,
- 2 each C_i yields C_{i+1} , and
- 3 C_n is an accepting configuration.

The *language* $L(M)$ of M is the set of strings accepted by M .

Example Turing Machine

Example

A Turing Machine that accepts $\{ww : w \in \{0,1\}^*\}$.

Informal description:

- 1 Place endmarkers $\vdash, \dashv$ at either end of the input
- 2 Check the input is of even length – reject if it is not
- 3 Take the leftmost symbol that is a 0 or 1: Replace 0 with a and 1 with b
- 4 Take the rightmost symbol that is a 0 or 1: Replace 0 with A and 1 with B
- 5 Repeat steps 3 and 4 until there are no more 0's or 1's
- 6 Take the leftmost symbol that is an a or b : Erase that symbol, but use the state to remember it
- 7 Take the leftmost symbol that is an A or B : Reject if the symbol does not match the state, otherwise erase the symbol
- 8 Repeat steps 6 and 7 until there are no more symbols
- 9 Accept the input

Example Turing Machine

Example

A Turing Machine that accepts $\{ww : w \in \{0,1\}^*\}$.

Formal description

$(\{q_0, \dots, q_{12}, q_{\text{accept}}, q_{\text{reject}}\}, \{0, 1\}, \{\sqcup, 0, 1, a, b, A, B, \vdash, \neg\}, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$

where δ is defined as:

	$\sqcup$	0	1	a	b	A	B	$\vdash$	$\neg$
q_0	q_{accept}	$(q_1, 0, L)$	$(q_1, 1, L)$						
q_1	$(q_2, \vdash, R)$								
q_2	$(q_4, \neg, L)$	$(q_3, 0, R)$	$(q_3, 1, R)$						
q_3	q_{reject}	$(q_2, 0, R)$	$(q_2, 1, R)$						
q_4		(q_5, A, L)	(q_5, B, L)	(q_8, a, L)	(q_8, b, L)				
q_5		$(q_5, 0, L)$	$(q_5, 1, L)$	(q_6, a, R)	(q_6, b, R)			$(q_6, \vdash, R)$	
q_6		(q_7, a, R)	(q_7, b, R)						
q_7		$(q_7, 0, R)$	$(q_7, 1, R)$			(q_4, A, L)	(q_4, B, L)		
q_8	$(q_8, \sqcup, L)$			(q_8, a, L)	(q_8, b, L)	(q_8, A, L)	(q_8, B, L)	$(q_9, \vdash, R)$	
q_9	$(q_9, \sqcup, R)$			$(q_{10}, \sqcup, R)$	$(q_{11}, \sqcup, R)$				q_{accept}
q_{10}	$(q_{10}, \sqcup, R)$			(q_{10}, a, R)	(q_{10}, b, R)	$(q_8, \sqcup, L)$	q_{reject}		q_{reject}
q_{11}	$(q_{11}, \sqcup, R)$			(q_{11}, a, R)	(q_{11}, b, R)	q_{reject}	$(q_8, \sqcup, L)$		q_{reject}

Example Turing Machine

Example

A Turing Machine that accepts $\{ww : w \in \{0,1\}^*\}$.

	$q_0 0101$	$\rightarrow$	$q_1 \sqcup 0101$	$\rightarrow$	$\vdash q_2 0101$
$\rightarrow$	$\vdash 0q_3 101$	$\rightarrow$	$\vdash 01q_2 01$	$\rightarrow$	$\vdash 010q_3 1$
$\rightarrow$	$\vdash 0101q_2 \sqcup$	$\rightarrow$	$\vdash 010q_4 1 \dashv$	$\rightarrow$	$\vdash 01q_5 0B \dashv$
$\rightarrow$	$\vdash 0q_5 10B \dashv$	$\rightarrow$	$\vdash q_5 010B \dashv$	$\rightarrow$	$q_5 \vdash 010B \dashv$
$\rightarrow$	$\vdash q_6 010B \dashv$	$\rightarrow$	$\vdash aq_7 10B \dashv$	$\rightarrow$	$\vdash a1q_7 0B \dashv$
$\rightarrow$	$\vdash a10q_7 B \dashv$	$\rightarrow$	$\vdash a1q_4 0B \dashv$	$\rightarrow$	$\vdash aq_5 1AB \dashv$
$\rightarrow$	$\vdash q_5 a1AB \dashv$	$\rightarrow$	$\vdash aq_6 1AB \dashv$	$\rightarrow$	$\vdash abq_7 AB \dashv$
$\rightarrow$	$\vdash aq_4 bAB \dashv$	$\rightarrow$	$\vdash q_8 abAB \dashv$	$\rightarrow$	$q_8 \vdash abAB \dashv$
$\rightarrow$	$\vdash q_9 abAB \dashv$	$\rightarrow$	$\vdash \sqcup q_{10} bAB$	$\rightarrow$	$\vdash \sqcup bq_{10} AB \dashv$
$\rightarrow$	$\vdash \sqcup q_8 b \sqcup B \dashv$	$\rightarrow$	$\vdash q_8 \sqcup b \sqcup B \dashv$	$\rightarrow$	$q_8 \vdash \sqcup b \sqcup B \dashv$
$\rightarrow$	$\vdash q_9 \sqcup b \sqcup B \dashv$	$\rightarrow$	$\vdash \sqcup q_9 b \sqcup B \dashv$	$\rightarrow$	$\vdash \sqcup \sqcup q_{11} \sqcup B \dashv$
$\rightarrow$	$\vdash \sqcup \sqcup \sqcup q_{11} B \dashv$	$\rightarrow$	$\vdash \sqcup \sqcup q_8 \sqcup \sqcup \dashv$	$\rightarrow$	$\vdash \sqcup q_8 \sqcup \sqcup \sqcup \dashv$
$\rightarrow$	$\vdash q_8 \sqcup \sqcup \sqcup \sqcup \dashv$	$\rightarrow$	$q_8 \vdash \sqcup \sqcup \sqcup \sqcup \dashv$	$\rightarrow$	$\vdash \sqcup q_9 \sqcup \sqcup \sqcup \dashv$
$\rightarrow$	$\vdash \sqcup \sqcup q_9 \sqcup \sqcup \dashv$	$\rightarrow$	$\vdash \sqcup \sqcup \sqcup q_9 \sqcup \dashv$	$\rightarrow$	$\vdash \sqcup \sqcup \sqcup \sqcup q_9 \dashv \rightarrow q_{\text{accept}}$

Example Turing Machine

Example

A Turing Machine that accepts $\{ww : w \in \{0,1\}^*\}$.

Implementation at <http://morphett.info/turing/turing.html>

Recursively Enumerable and Recursive Languages

Definition

$L \in \Sigma^*$ is *Turing-recognisable* (or *recursively enumerable*, *semi-decidable*) if some TM M recognises it, i.e. $L = L(M)$.

A TM can fail to accept some input either by:

- *rejecting*: eventually entering q_{reject} , or
- *diverging*: entering an infinite loop.

Divergence causes trouble. TMs that never diverge are called *deciders*.

Definition

$L \in \Sigma^*$ is (*Turing-*)*decidable* (or *recursive*) if some TM decides it.

Semi-Decidable vs Decidable

A more algorithmic version of the previous definitions:

- L is **RE** (or **semi-decidable**) if there is a TM such that:
 - For every $w \in L$, the TM will accept w in finite time
- L is **recursive** (or **decidable**) if there is a TM such that:
 - For every $w \in L$, the TM will accept w in finite time, AND
 - For every $w \notin L$, the TM will reject w in finite time.

Outline

History

Turing Machines

Turing Machine Variants

Enumeration

Closure Properties

Encodings

The Universal Turing Machine

Stay put TMs

So far, our TMs had to move left or right in each step. Suppose we add a 3rd option—to stay put, so the type of a TM's transition function becomes:

$$\delta : Q \times \Gamma \longrightarrow Q \times \Gamma \times \{L, R, S\}$$

Stay put TMs

So far, our TMs had to move left or right in each step. Suppose we add a 3rd option—to stay put, so the type of a TM's transition function becomes:

$$\delta : Q \times \Gamma \longrightarrow Q \times \Gamma \times \{L, R, S\}$$

Theorem

Every stay put TM has an equivalent ordinary TM.

Stay put TMs

So far, our TMs had to move left or right in each step. Suppose we add a 3rd option—to stay put, so the type of a TM's transition function becomes:

$$\delta : Q \times \Gamma \longrightarrow Q \times \Gamma \times \{L, R, S\}$$

Theorem

Every stay put TM has an equivalent ordinary TM.

Proof.

Replace each S transition with two transitions: R followed by L . □

As we shall see, the TM model is fairly *robust* when it comes to variations and extensions.

Sipser's TM

Sipser's TMs work with a *left-bounded tape*. The head starts on the left-most square of the tape with the head on the first letter of the input.

Should the transition function suggest to move left when the head is already at the left-most position, the head stays put.

Theorem

Every doubly infinite tape TM has an equivalent left-bounded TM.

Sipser's TM

Sipser's TMs work with a *left-bounded tape*. The head starts on the left-most square of the tape with the head on the first letter of the input.

Should the transition function suggest to move left when the head is already at the left-most position, the head stays put.

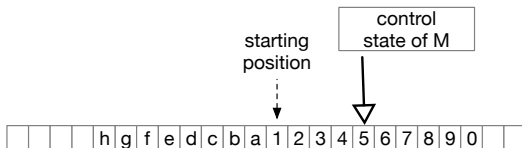
Theorem

Every doubly infinite tape TM has an equivalent left-bounded TM.

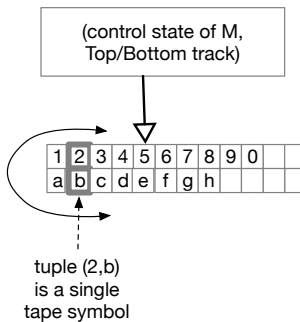
Proof idea.

To simulate a doubly infinite tape TM by a left-bounded one, we split the left-bounded tape into an upper and a lower half. The upper half represents the right half of the tape (from the initial head position onwards) and the lower half represents the left half. An extra state component keeps track of the half in which the head currently is. □

doubly
infinite tape



simulation using
left-bounded
infinite tape



Multitape TMs

A *multitape TM* has multiple tapes, each with its own head for reading and writing. Initially the input appears on tape 1, and the others start out blank. The type of the transition function of a k -tape TM becomes:

$$\delta : Q \times \Gamma^k \longrightarrow Q \times \Gamma^k \times \{L, R, S\}^k$$

Multitape TMs

A *multitape TM* has multiple tapes, each with its own head for reading and writing. Initially the input appears on tape 1, and the others start out blank. The type of the transition function of a k -tape TM becomes:

$$\delta : Q \times \Gamma^k \longrightarrow Q \times \Gamma^k \times \{L, R, S\}^k$$

Theorem

Every multitape TM has an equivalent single-tape TM.

[Sipser] has a proof but here's a different one inspired by [Hopcroft, Motwani & Ullman].

Multitape $\rightarrow$ Single tape

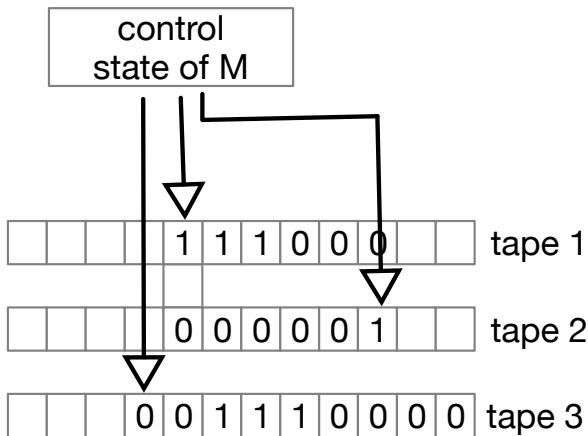
Proof idea.

Consider a k -tape TM $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$. Simulate M with a single-tape TM $N = (Q', \Sigma', \Gamma', \delta', q'_0, q'_{\text{accept}}, q'_{\text{reject}})$ where the crux is that the single tape of N is split into $2k$ tracks, half of which hold the tapes of M and the other half marks the positions of the k heads:

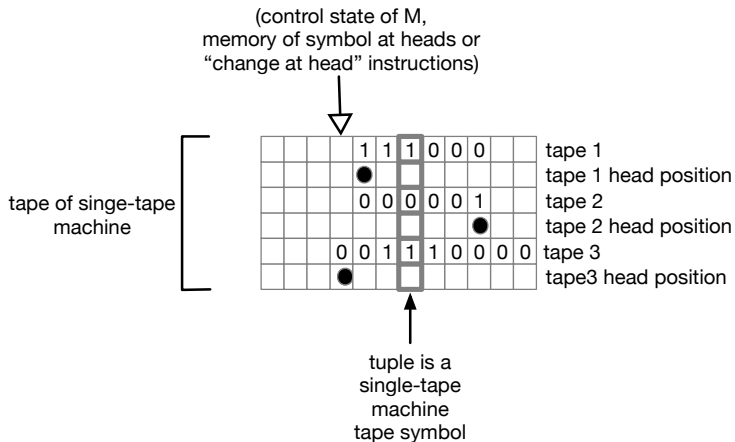
$$\Gamma' = (\Gamma \times \{\sqcup, \bullet\})^k$$

To simulate a move of M , N 's head must visit the k head markers. To avoid getting lost, it must remember (in its state) how many of the markers are to its left. After visiting all head markers and storing the scanned k symbols (again in its state) N knows what M would do based on the k symbols and M 's current state (stored again). So it does that by revisiting the k heads and updating the tracks and state. N 's accepting (rejecting) state is entered via a $\S$ transition whenever the stored state component of the simulated M is q_{accept} (q_{reject}). □

Multitape $\rightarrow$ Single tape



Multitape $\rightarrow$ Single tape



Non-Deterministic TMs

The type of the transition function of a non-deterministic TM becomes:

$$\delta : Q \times \Gamma \longrightarrow 2^{Q \times \Gamma \times \{L,R,S\}}$$

Non-Deterministic TMs

The type of the transition function of a non-deterministic TM becomes:

$$\delta : Q \times \Gamma \longrightarrow 2^{Q \times \Gamma \times \{L,R,S\}}$$

Theorem

Every non-deterministic TM has an equivalent deterministic TM.

Non-Deterministic TMs

The type of the transition function of a non-deterministic TM becomes:

$$\delta : Q \times \Gamma \longrightarrow 2^{Q \times \Gamma \times \{L,R,S\}}$$

Theorem

Every non-deterministic TM has an equivalent deterministic TM.

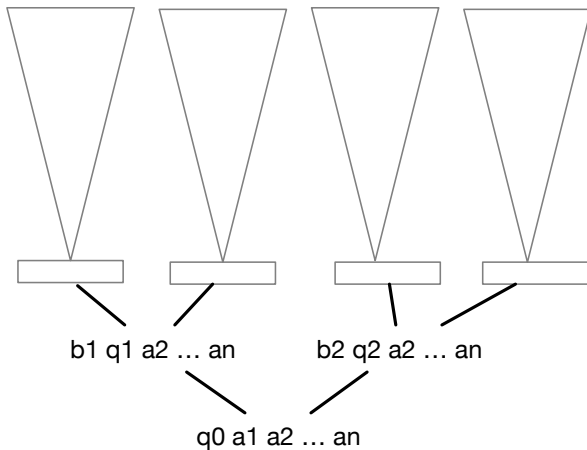
[A variant of this theorem holds for deciders.]

Non-deterministic $\rightarrow$ Deterministic

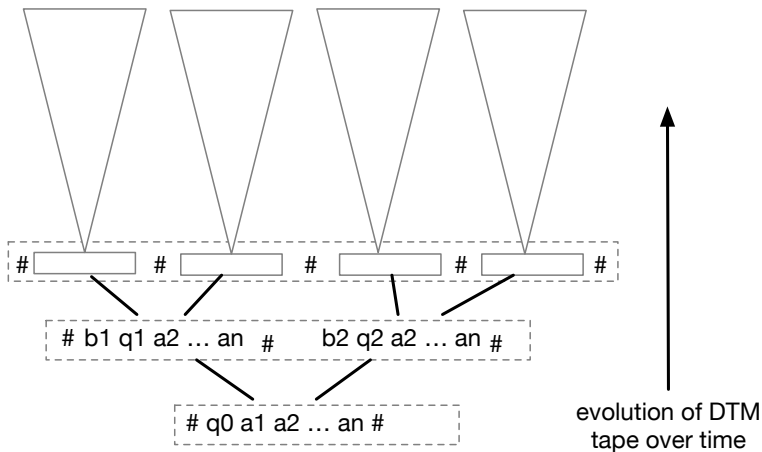
Proof idea.

We simulate any non-deterministic TM N with a deterministic TM D that breadth-first searches for q_{accept} on all possible branches of N 's non-deterministic computation. If D ever finds q_{accept} on one of these branches, it accepts. Otherwise it will diverge. $\square$

A computation tree of a non-deterministic machine



Deterministic simulation of the computation tree of a non-deterministic machine



Outline

History

Turing Machines

Turing Machine Variants

Enumeration

Closure Properties

Encodings

The Universal Turing Machine

Enumeration

Going forward, we will see that enumeration is a very useful technique, especially when describing semi-decidable procedures.

For example:

- Try all words in Σ^* in increasing length (i.e. lenlex order)
- Run through all computation paths with a BFS
- For x in $\langle \text{enumerable set} \rangle$ (note: countable is not good enough)

Enumerators

An enumerator is a TM-like construct that starts with a blank input tape and, if it doesn't halt, may print an infinite list of strings onto the tape. It does so by every now and then entering a *print state* which means that the terminal string to the right of the head (incl. what's under the head) is printed/enumerated.

The language of an enumerator is the set of all strings it eventually prints out. (It may print strings repeatedly and in any order.)

Enumerator examples

Examples

Enumerating the prime numbers:

- ➊ Starting with $n = 2$ on the tape
- ➋ Check if n is divisible by anything other than 1 and n .
If not, print n
- ➌ Increment n
- ➍ Return to step 2

Enumerator examples

Examples

Enumerating all Turing Machines (fixed Σ and Γ):

- ➊ Starting with $|Q| = \{q_{\text{accept}}, q_{\text{reject}}\}$
- ➋ For all possible transition relations $\delta : Q \times \Gamma \rightarrow (Q \times \Gamma \times \{L, R\})$ and all possible start states $q_0 \in Q$:
 - Print $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$
- ➌ Add a new state to Q
- ➍ Return to step 2.

Dovetailing

Enumerating infinitely many enumerations

	t_1	t_2	t_3	t_4	$\dots$
M_1	•	•	•	•	$\dots$
M_2					
M_3					
M_4					
$\vdots$					

Dovetailing

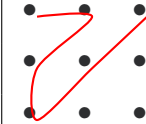
Enumerating infinitely many enumerations

	t_1	t_2	t_3	t_4	$\dots$
M_1	•	•	•	•	$\dots$
M_2	•	•	•	•	$\dots$
M_3	•	•	•	•	$\dots$
M_4	•	•	•	•	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Dovetailing

Enumerating infinitely many enumerations

	t_1	t_2	t_3	t_4	$\dots$
M_1	•	•	•	•	$\dots$
M_2	•	•	•	•	$\dots$
M_3	•	•	•	•	$\dots$
M_4	•	•	•	•	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$



Enumerators $\equiv$ Turing Machines

The origin of the term *recursively enumerable* for Turing-recognisable is

Theorem

A language is Turing-recognisable iff some enumerator enumerates it.

Enumerators $\equiv$ Turing Machines

The origin of the term *recursively enumerable* for Turing-recognisable is

Theorem

A language is Turing-recognisable iff some enumerator enumerates it.

Proof idea.

“ $\Leftarrow$ ”: Let E be an enumerator. The TM M_E works as follows. On input w it runs E and, each time E prints a word v , compares w with v and accepts if they're equal.

Enumerators $\equiv$ Turing Machines

The origin of the term *recursively enumerable* for Turing-recognisable is

Theorem

A language is Turing-recognisable iff some enumerator enumerates it.

Proof idea.

“ $\Leftarrow$ ”: Let E be an enumerator. The TM M_E works as follows. On input w it runs E and, each time E prints a word v , compares w with v and accepts if they're equal.

“ $\Rightarrow$ ”: Let M be a TM. Let $(w_i)_{i \in \mathbb{N}}$ be an enumeration of Σ^* . The enumerator E_M works as follows. Repeat for $i = 1, 2, \dots$ to run M for i steps on each of the first i words $w_0, \dots, w_{i-1}$ in the enumeration. If i -step computation on w_j accepts, print w_j . $\square$

Outline

History

Turing Machines

Turing Machine Variants

Enumeration

Closure Properties

Encodings

The Universal Turing Machine

Closure properties of (Semi-)Decidable Languages

Theorem

The (semi-)decidable languages are closed under union.

Proof.

Let L_1 and L_2 be languages that are decided (accepted) by TMs M_1 and M_2 respectively. We construct a TM that decides (accepts) $L_1 \cup L_2$ as follows.

On input x :

- 1 Run M_1 and M_2 in parallel on x .
- 2 Accept if either M_1 or M_2 accepts.



Closure properties of (Semi-)Decidable Languages

Theorem

The decidable languages are closed under complement.

Proof.

Suppose L is decided by M . Define M' to be the same as M with q_{accept} and q_{reject} swapped. M' decides $\Sigma^* \setminus L$. □

Corollary

The decidable languages are closed under intersection.

Closure properties of (Semi-)Decidable Languages

Theorem

The decidable languages are closed under complement.

Proof.

Suppose L is decided by M . Define M' to be the same as M with q_{accept} and q_{reject} swapped. M' decides $\Sigma^* \setminus L$. □

Corollary

The decidable languages are closed under intersection.

Question

What about semi-decidable languages?

Rec, RE, and coRE

Definition

A language $L \in \Sigma^*$ is in coRE if $\bar{L} = \Sigma^* \setminus L$ is in RE.

NB

$L \in \text{coRE}$ if there is a Turing Machine such that:

- If $w \notin L$ then the the TM will reject w in finite time.

$$\text{Rec} = \text{RE} \cap \text{coRE}$$

Theorem

A language is decidable iff it and its complement are RE. That is,

$$\text{Rec} = \text{RE} \cap \text{coRE}.$$

$$\text{Rec} = \text{RE} \cap \text{coRE}$$

Theorem

A language is decidable iff it and its complement are RE. That is,

$$\text{Rec} = \text{RE} \cap \text{coRE}.$$

Proof.

$\subseteq$: follows from the closure properties of decidable languages.

$\supseteq$: We construct a decider D for L from a TM M that accepts L and a TM M' that accepts $\bar{L}$ as follows.

On input x :

- 1 Run M and M' on x in parallel.
- 2 If M accepts, then accept.
- 3 If M' accepts, then reject.

Outline

History

Turing Machines

Turing Machine Variants

Enumeration

Closure Properties

Encodings

The Universal Turing Machine

Encodings

The input to a Turing Machine is always a string, but it can be useful to work with objects other than words over Σ .

If O is a (finitely-presented) object, an **encoding** of O (over Σ), $\langle O \rangle$, is a representation of O as a string.

NB

The exact details of the encoding are unimportant (here).

Exception

One exception (important later) is the encoding of numbers:

Unary: 0^n vs Binary: $\text{bin}(n)$

Encodings: Example

Example

We can encode Turing Machines as strings over $\{0, 1\}$.

E.g. if $\langle M \rangle$ begins with the prefix

$$0^n 1 0^m 1 0^k 1 0^s 1 0^t 1 0^r 1 0^u$$

this might indicate that M has n states $(0, 1, \dots, n-1)$; it has m tape symbols $(0, 1, \dots, m-1)$, of which the first k are input symbols; the start, accept and reject states are s , t and r respectively, and the blank symbol is u . The remainder of the string can consist of a sequence of substrings specifying the transitions in δ .

E.g. $0^p 1 0^a 1 0^q 1 0^b 1 0$ might indicate that δ contains the transition

$$((p, a), (q, b, L)).$$

Outline

History

Turing Machines

Turing Machine Variants

Enumeration

Closure Properties

Encodings

The Universal Turing Machine

The Universal Turing Machine

A **universal** TM is a TM U that can simulate the actions of any Turing machine. I.e. for any TM M and any input x of M , U accepts (respectively rejects or loops on) $\langle M, x \rangle$, if and only if, M accepts (respectively rejects or loops on) x .

This is equivalent to an interpreter of C written in C.

Note this is different (quantifier order) from results such as every NDTM can be simulated by a deterministic TM.

Universal Turing Machine

Simulation. The tape of U is partitioned into 3 tracks:

- The top track holds the transition function δ_M of the input TM M .
- The middle track will be used to hold the simulated contents of M 's tape.
- The bottom track will hold the current state of M , and the current position of M 's tape head.

U simulates M on input x one step at a time, shuttling back and forth between the three tracks. In each step, it updates M 's state and simulated tape contents and head position as dictated by δ_M . If ever M halts and accepts or halts and rejects, then U does the same.

Note: The input alphabet of M and U may be different.